Module 3 – Exercise 1: Ranking and Window Functions

Aim

To understand and use SQL Server Window Functions for ranking, numbering, and performing calculations on rows without grouping the data.

Theory

Window functions perform calculations across a set of rows while keeping each row separate. They are commonly used for ranking employees, finding top records, calculating running totals, and comparing values.

OVER()

The **OVER()** clause defines the window or set of rows on which the function operates. It is mandatory for all window functions.

PARTITION BY

The **PARTITION BY** clause divides the result into groups. Each group is processed independently by the window function.

ROW_NUMBER()

Assigns a unique sequential number to each row. Even if two rows have the same value, different row numbers are assigned.

RANK()

Assigns the same rank to rows with equal values. The next rank is skipped after duplicate ranks.

DENSE_RANK()

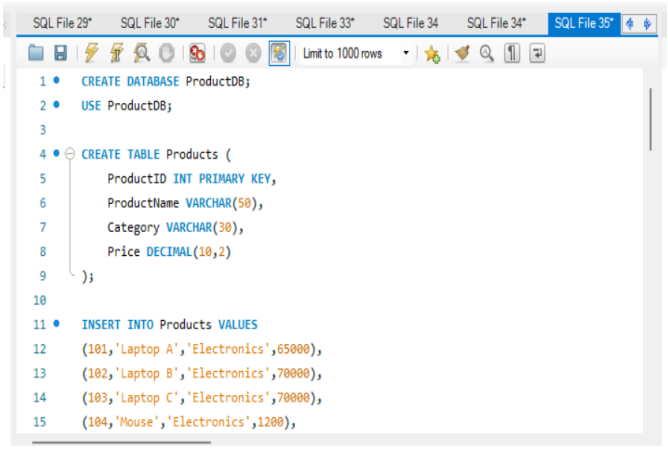
Assigns the same rank to duplicate values but does not skip the next rank.

Advantages of Window Functions

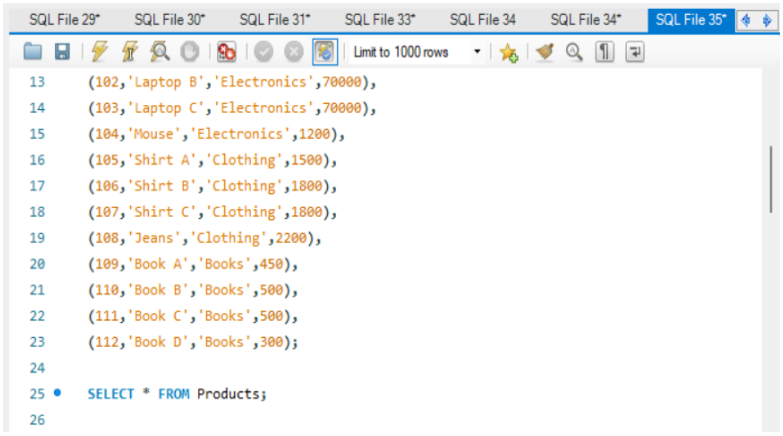
- Simplifies complex SQL queries.
- Useful for ranking and sorting data.
- Calculates running totals and averages.
- Improves query readability.

CODE :

Created the product Datebase :

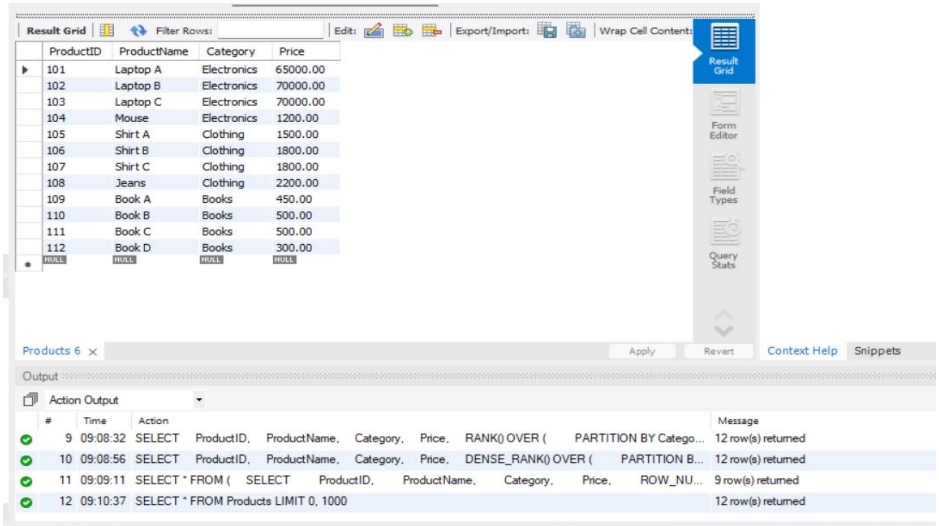


```
CREATE DATABASE ProductDB;
USE ProductDB;

CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Category VARCHAR(30),
    Price DECIMAL(10,2)
);

INSERT INTO Products VALUES
(101,'Laptop A','Electronics',65000),
(102,'Laptop B','Electronics',70000),
(103,'Laptop C','Electronics',70000),
(104,'Mouse','Electronics',1200),
```



```
(102,'Laptop B','Electronics',70000),
(103,'Laptop C','Electronics',70000),
(104,'Mouse','Electronics',1200),
(105,'Shirt A','Clothing',1500),
(106,'Shirt B','Clothing',1800),
(107,'Shirt C','Clothing',1800),
(108,'Jeans','Clothing',2200),
(109,'Book A','Books',450),
(110,'Book B','Books',500),
(111,'Book C','Books',500),
(112,'Book D','Books',300);

SELECT * FROM Products;
```

OUTPUT :



Result Grid

ProductID	ProductName	Category	Price
101	Laptop A	Electronics	65000.00
102	Laptop B	Electronics	70000.00
103	Laptop C	Electronics	70000.00
104	Mouse	Electronics	1200.00
105	Shirt A	Clothing	1500.00
106	Shirt B	Clothing	1800.00
107	Shirt C	Clothing	1800.00
108	Jeans	Clothing	2200.00
109	Book A	Books	450.00
110	Book B	Books	500.00
111	Book C	Books	500.00
112	Book D	Books	300.00

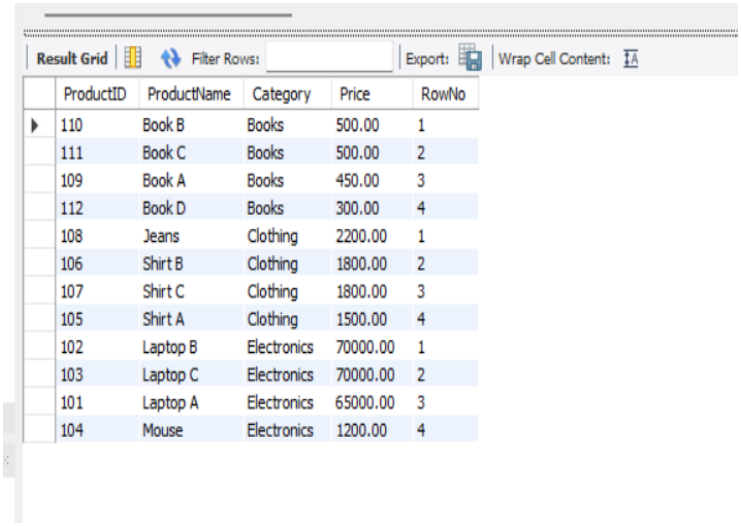
Output

#	Time	Action	Message
9	09:08:32	SELECT ProductID, ProductName, Category, Price, RANK() OVER (PARTITION BY Catego...	12 row(s) returned
10	09:08:56	SELECT ProductID, ProductName, Category, Price, DENSE_RANK() OVER (PARTITION B...	12 row(s) returned
11	09:09:11	SELECT * FROM (SELECT ProductID, ProductName, Category, Price, ROW_NU...	9 row(s) returned
12	09:10:37	SELECT * FROM Products LIMIT 0, 1000	12 row(s) returned

1. ROW_NUMBER()

```
26
27 • SELECT
28       ProductID,
29       ProductName,
30       Category,
31       Price,
32       ROW_NUMBER() OVER (
33           PARTITION BY Category
34           ORDER BY Price DESC
35       ) AS RowNo
36   FROM Products;
37
```

OUTPUT :



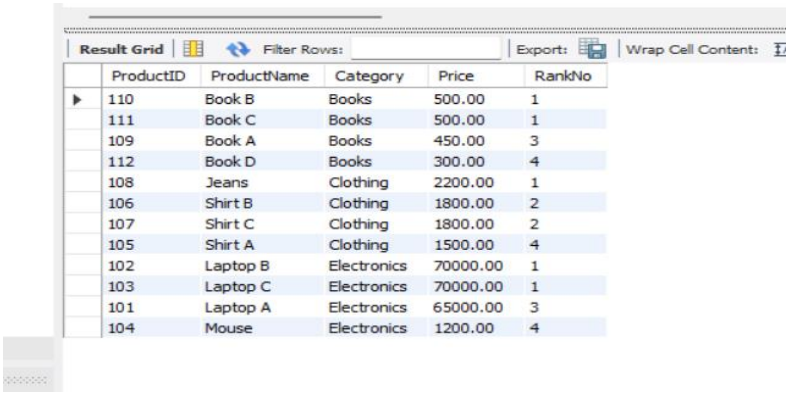
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	ProductName	Category	Price	RowNo
110	Book B	Books	500.00	1
111	Book C	Books	500.00	2
109	Book A	Books	450.00	3
112	Book D	Books	300.00	4
108	Jeans	Clothing	2200.00	1
106	Shirt B	Clothing	1800.00	2
107	Shirt C	Clothing	1800.00	3
105	Shirt A	Clothing	1500.00	4
102	Laptop B	Electronics	70000.00	1
103	Laptop C	Electronics	70000.00	2
101	Laptop A	Electronics	65000.00	3
104	Mouse	Electronics	1200.00	4

2. RANK()

```
37
38 • SELECT
39       ProductID,
40       ProductName,
41       Category,
42       Price,
43       RANK() OVER (
44           PARTITION BY Category
45           ORDER BY Price DESC
46       ) AS RankNo
47   FROM Products;
48
```

OUTPUT:

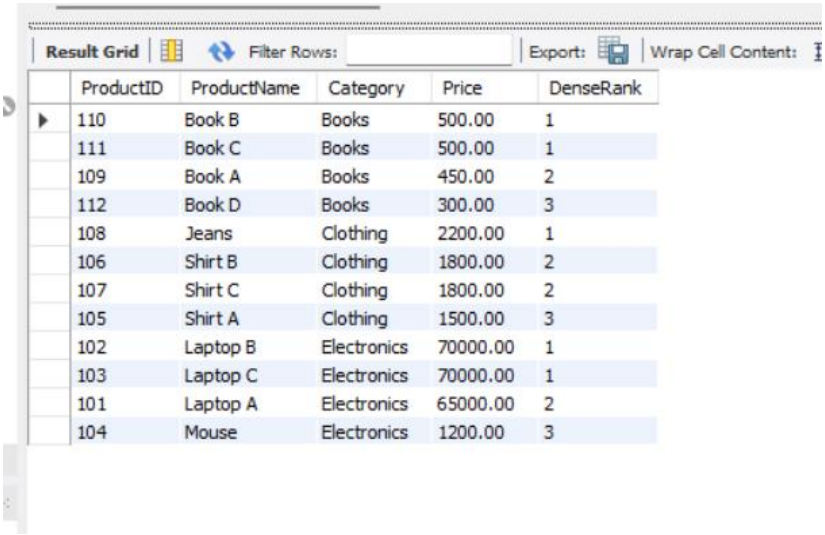


ProductID	ProductName	Category	Price	RankNo
110	Book B	Books	500.00	1
111	Book C	Books	500.00	1
109	Book A	Books	450.00	3
112	Book D	Books	300.00	4
108	Jeans	Clothing	2200.00	1
106	Shirt B	Clothing	1800.00	2
107	Shirt C	Clothing	1800.00	2
105	Shirt A	Clothing	1500.00	4
102	Laptop B	Electronics	70000.00	1
103	Laptop C	Electronics	70000.00	1
101	Laptop A	Electronics	65000.00	3
104	Mouse	Electronics	1200.00	4

3. DENSE_RANK()

```
48
49 •  SELECT
50        ProductID,
51        ProductName,
52        Category,
53        Price,
54        DENSE_RANK() OVER (
55            PARTITION BY Category
56            ORDER BY Price DESC
57        ) AS DenseRank
58    FROM Products;
59
```

OUTPUT :

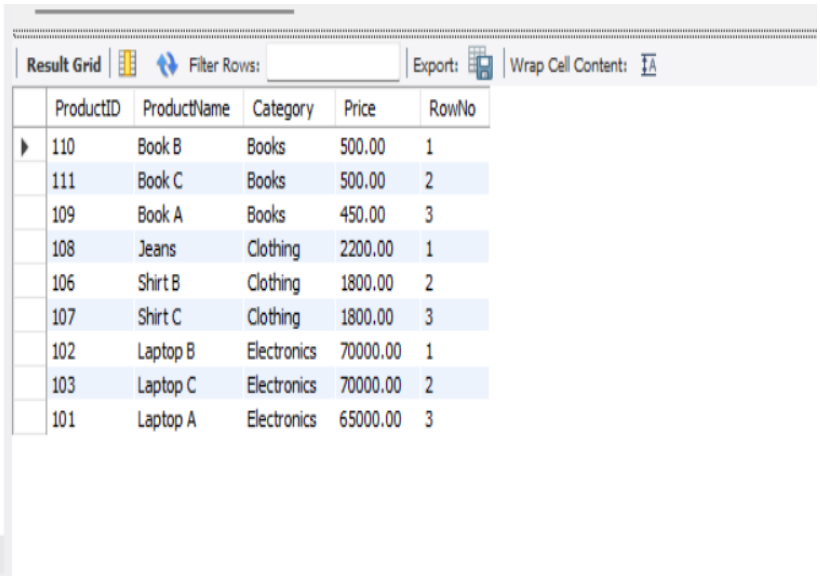


ProductID	ProductName	Category	Price	DenseRank
110	Book B	Books	500.00	1
111	Book C	Books	500.00	1
109	Book A	Books	450.00	2
112	Book D	Books	300.00	3
108	Jeans	Clothing	2200.00	1
106	Shirt B	Clothing	1800.00	2
107	Shirt C	Clothing	1800.00	2
105	Shirt A	Clothing	1500.00	3
102	Laptop B	Electronics	70000.00	1
103	Laptop C	Electronics	70000.00	1
101	Laptop A	Electronics	65000.00	2
104	Mouse	Electronics	1200.00	3

4. Top 3 Most Expensive Products in Each Category

```sql
SELECT *
FROM (
    SELECT
        ProductID,
        ProductName,
        Category,
        Price,
        ROW_NUMBER() OVER (
            PARTITION BY Category
            ORDER BY Price DESC
        ) AS RowNo
    FROM Products
) AS T
WHERE RowNo <= 3;
```

OUTPUT :



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	ProductName	Category	Price	RowNo
110	Book B	Books	500.00	1
111	Book C	Books	500.00	2
109	Book A	Books	450.00	3
108	Jeans	Clothing	2200.00	1
106	Shirt B	Clothing	1800.00	2
107	Shirt C	Clothing	1800.00	3
102	Laptop B	Electronics	70000.00	1
103	Laptop C	Electronics	70000.00	2
101	Laptop A	Electronics	65000.00	3